Zen User Guide

Zen — User Guide

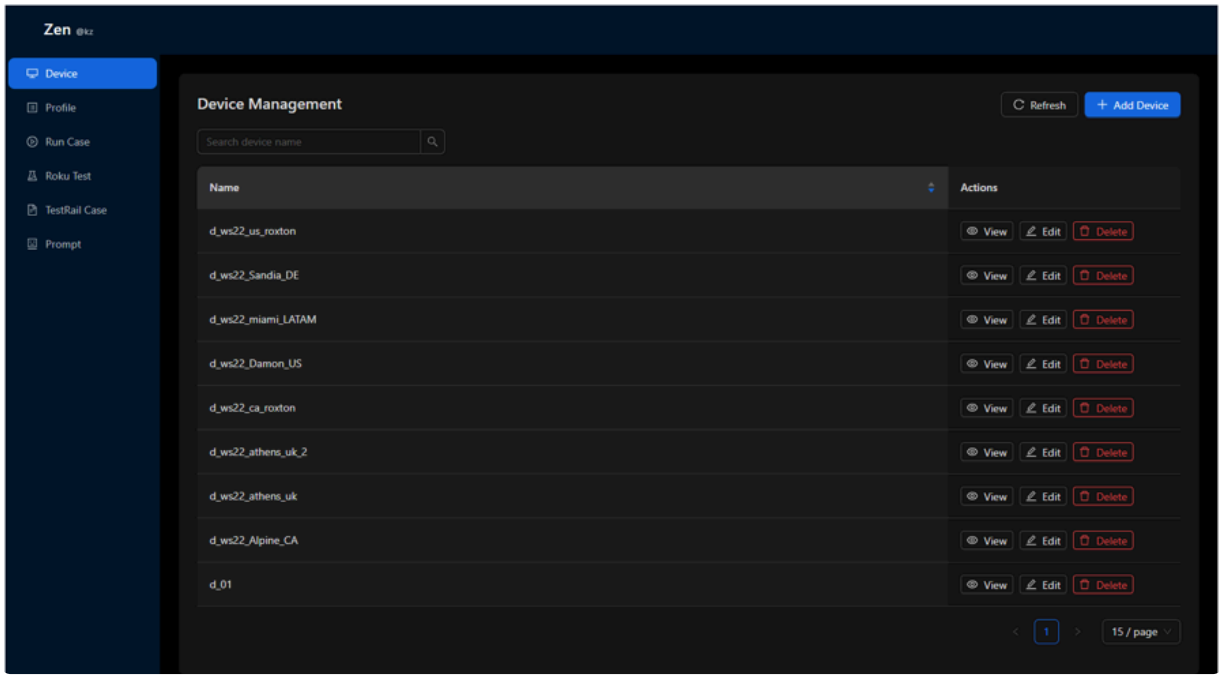
Zen is a web console for Roku QA / test-automation. It gives you a single dark-themed UI to manage device configurations, build and run test profiles, run individual Roku tests, browse TestRail results, and talk to an AI assistant that has access to Roku internal data (JIRA / Confluence / TestRail / GitLab).

> **Live instance:** http://10.64.16.16:9898/ | **Backend API:** proxied under `/api` to the FastAPI service on port 9888.

> Screenshots below are captured from the live instance. A few detail dialogs (Add Device, Skill editor, live log) still show an *[Screenshot: ...]* placeholder you can fill in later.

1. Getting Started

Open http://10.64.16.16:9898/ in a browser (Chrome/Edge recommended — see the note on Save/Copy below). No login is required.



1.1 Layout

- **Header bar** — dark bar at the top showing the brand **Zen** with a dim **@kz** superscript.
- **Left navigation (Sider)** — the tab menu. It collapses automatically on narrow screens. Click a tab to switch pages; the active tab is highlighted.

- **Content area** — a rounded card that shows the current page.

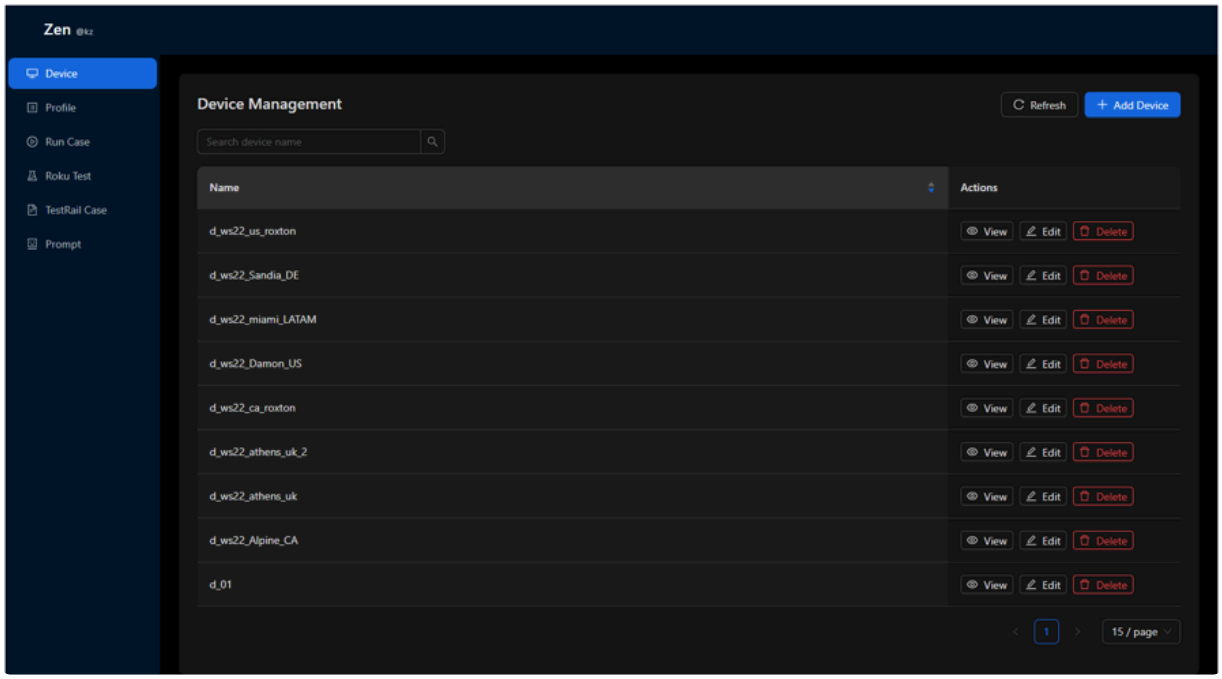
1.2 Navigation tabs

Tab	Purpose
Device	Manage device configuration files (the TVs / Roku devices under test).
Profile	Manage test profiles and generate them from TestRail data.
Run Case	Run test profiles, watch running processes and QA Hub suites, and review results.
Roku Test	Run a single Roku test with a live log tail, and browse past test logs.
TestRail Case	Manage TestRail plans and search their cases by status.
Prompt	AI assistant (RAG) over Roku internal data, plus reusable prompt Skills.

Any tab can be hidden by an administrator via `frontend/conf/config.yml` (the `tabs:` section). If a tab is missing, it was intentionally disabled in configuration.

2. Device Tab

The Device tab lists every device configuration file. Each device is a JSON file describing a TV/Roku under test (IP, brand, power strip, QA bot, serial console, etc.).



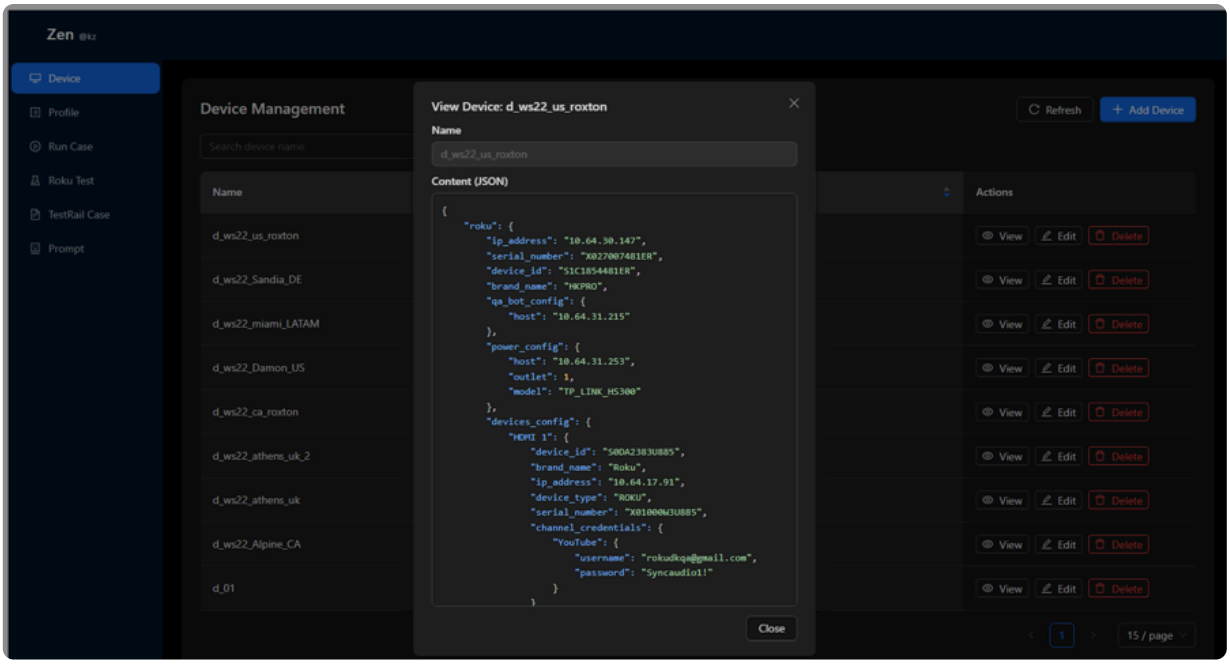
2.1 Browsing devices

- The table shows one **Name** per device (sortable; sorted descending by default).
- Use the **Search** box (top-left of the table) to filter by name.
- Paging controls are at the bottom (10 / 15 / 30 / 50 per page).
- Click **Refresh** (top-right) to reload the list from the server.

2.2 Viewing / editing a device

Each row has three actions:

- **View** — opens a read-only dialog showing the device JSON with **syntax highlighting**. The dialog width auto-sizes to the content.
- **Edit** — opens an editable JSON editor (also syntax-highlighted, live). Change the JSON and click **Save**; the content is validated as JSON before saving.
- **Delete** — removes the device after a confirmation prompt.



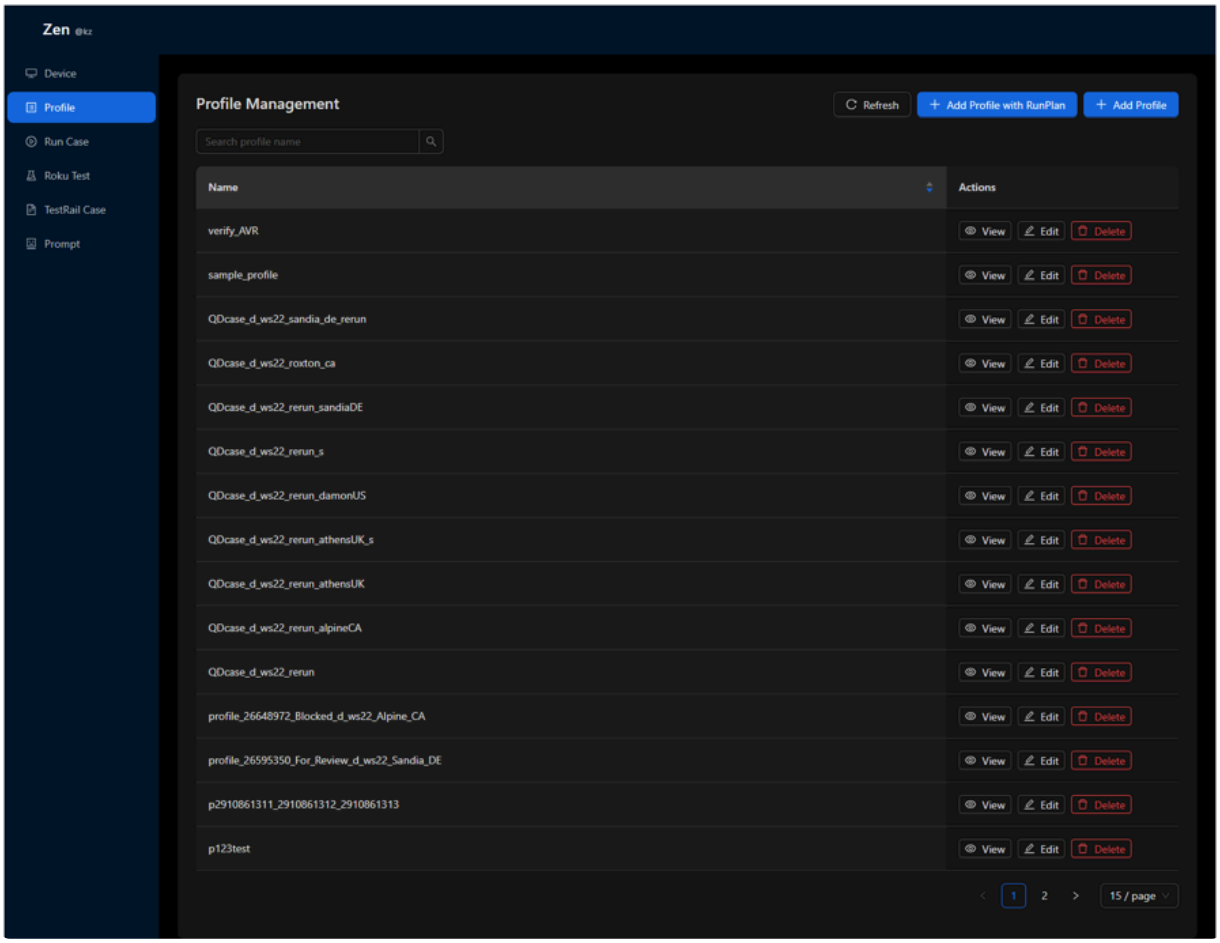
2.3 Adding a device

1. Click **Add Device** (top-right).
2. Enter a **Name** (without the `.json` extension) — this becomes the device key used in profiles, e.g. `d_ws22_Sandia_DE`.
3. Fill in the **Content (JSON)**. A minimal config needs `ip_address`, `brand_name`, and the device identifiers under a `roku` object.
4. Click **Save**.

[Screenshot: Add Device dialog — Name field and JSON editor with the starter template.]

3. Profile Tab

A **profile** is a JSON file that describes what to run: which cases, on which devices, in what order. The Profile tab manages these files and can also generate them automatically from TestRail data.



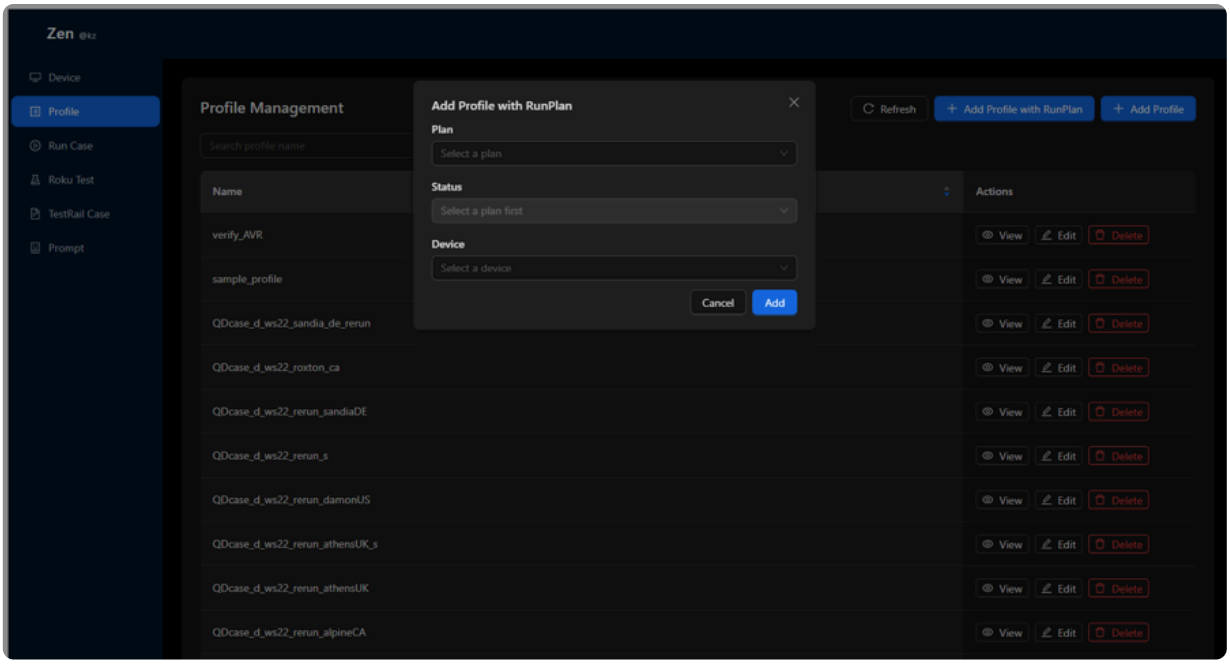
3.1 Managing profiles

Browsing, searching, View / Edit / Delete, and Add work exactly like the Device tab (same JSON editor with syntax highlighting and auto-sizing dialog).

3.2 Add Profile with RunPlan (generate from TestRail)

This is the fast way to build a re-run profile from failed/blocked/etc. cases already fetched from TestRail (see the TestRail Case tab, §6).

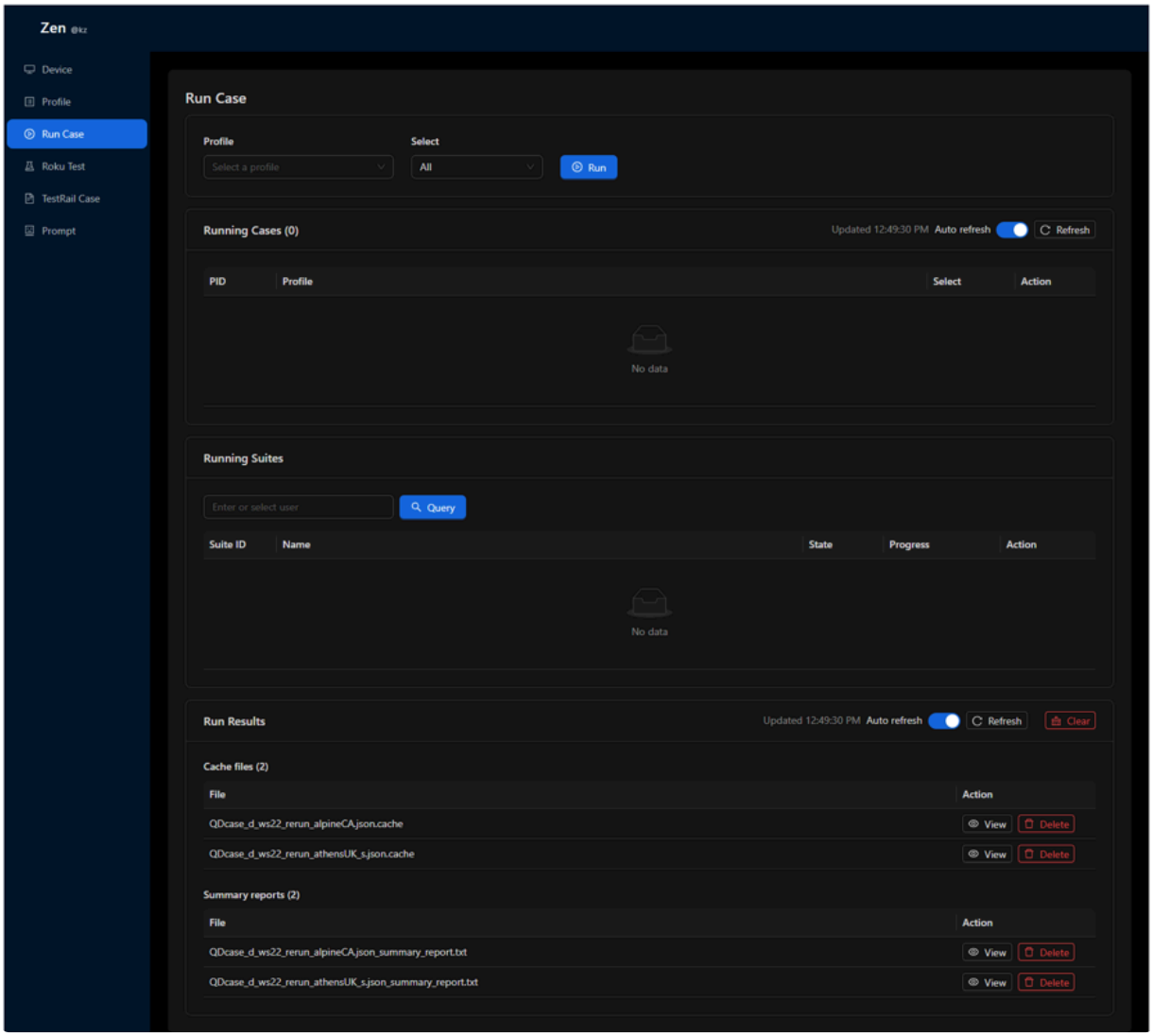
1. Click **Add Profile with RunPlan**.
2. **Plan** — pick a plan. The dropdown only lists plans that already have cached TestRail data, shown as *id - name*.
3. **Status** — pick a status. Only statuses that have cached data for the chosen plan appear (e.g. *Failed*). This is disabled until a plan is selected.
4. **Device** — pick the device to run the cases on.
5. Click **Add**. Zen builds a profile named `profile_<plan>_<status>_<device>.json` containing one task per case (each case's automation id becomes a BDD/UT config) and adds it to the profile list.



⚠ If a plan or status is not offered, its data has not been cached yet. Go to the **TestRail Case** tab and run a search for that plan+status first (§6.3).

4. Run Case Tab

The Run Case tab launches test profiles and gives you three live views: local runner processes, QA Hub suites, and result files. It auto-refreshes every few seconds when enabled.



4.1 Running a profile

1. **Profile** — select a profile (searchable).
2. **Select** — choose which result subset to run: *All, Fail, Pass, Skipped, Blocked, NA, Unknown, NoTest*.
3. Click **Run**. A confirmation banner shows the returned message and the process id (PID).

4.2 Running Cases (local processes)

Lists the `test_runner.py` processes currently running on the server. Columns: **PID**, **Profile**, **Select**, and an **Action** column with a **Stop** button (kills that process, with confirmation). Toggle **Auto refresh** or click **Refresh**; the header shows the last-updated time and a live count.

4.3 Running Suites (QA Hub)

1. Type a **user name** (autocomplete) and press Enter or click **Query**.
2. The table lists that user's running QA Hub suites: **Suite ID**, **Name** (links to the Jenkins build console), **State**, a **Progress** bar (tests complete / total), and a **Terminate** action.

[Screenshot: Running Suites card — a suite row with a progress bar and Terminate button.]

4.4 Run Results (cache & reports)

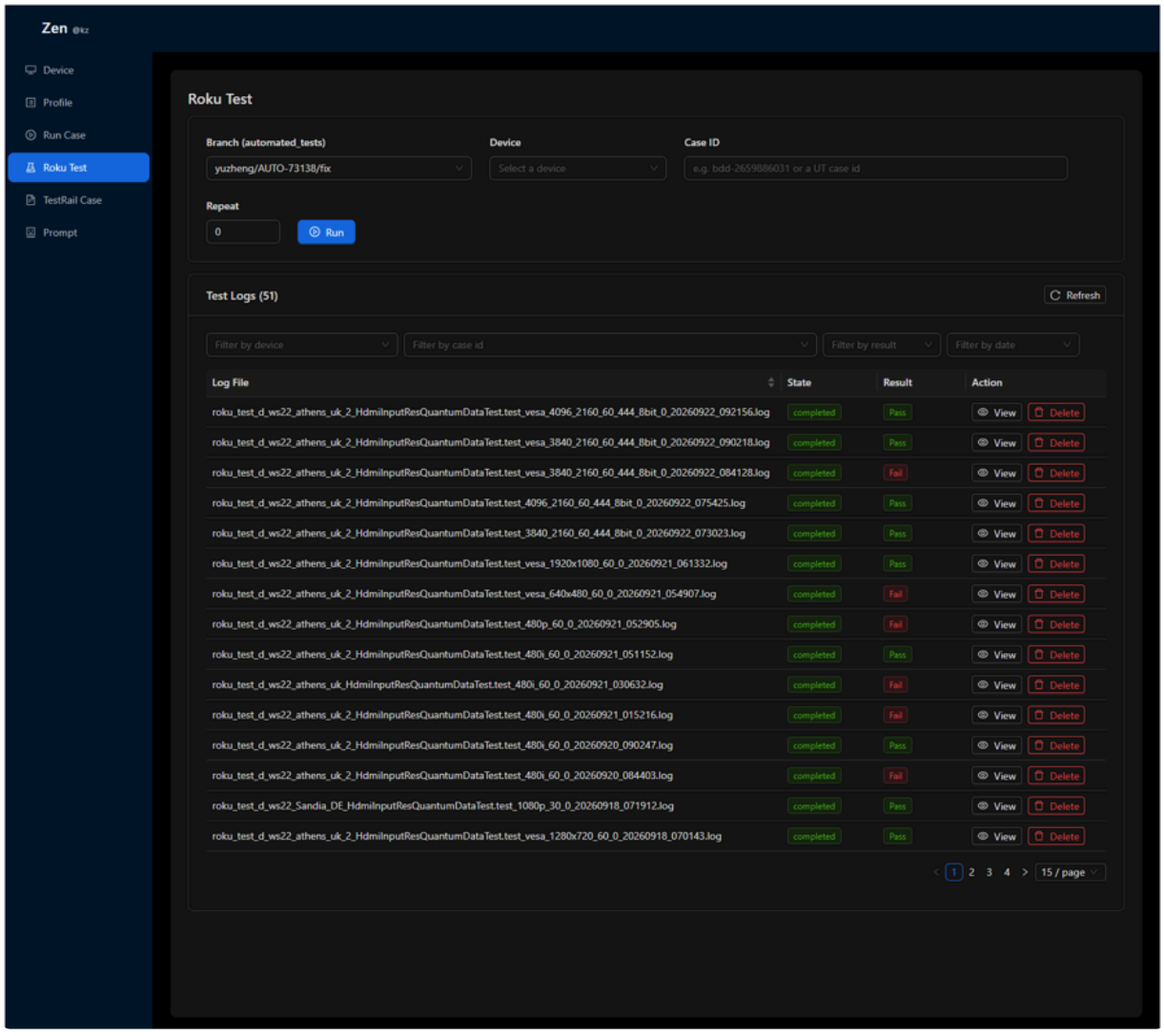
Two sub-tables list the runner's output files:

- **Cache files** — intermediate run state. **View** opens the raw file; **Delete** removes it.
- **Summary reports** — the human-readable run summary. Same View / Delete actions.

Use **Clear** (top-right of the card, with confirmation) to delete *all* cache and report files at once.

5. Roku Test Tab

Runs a single Roku test directly (via the `roku_test` CLI) and tails its log live. Useful for quick one-off verification on a specific branch and device.



5.1 Running a test

1. **Branch (automated_tests)** — pick the git branch. Selecting one checks it out on the server; the current branch is shown.

2. **Device** — pick the target device.
3. **Case ID** — enter the case, e.g. `bdd-2659886031` or a unit-test case id.
4. **Repeat** — how many extra repeats (0 = run once).
5. Click **Run**. A banner shows the PID, a live status tag (running / completed), and the exact command used.

5.2 Live log

The log panel streams the running test output, auto-scrolling. Error lines are highlighted red and warnings amber. The live view survives a page refresh as long as the test is still running.

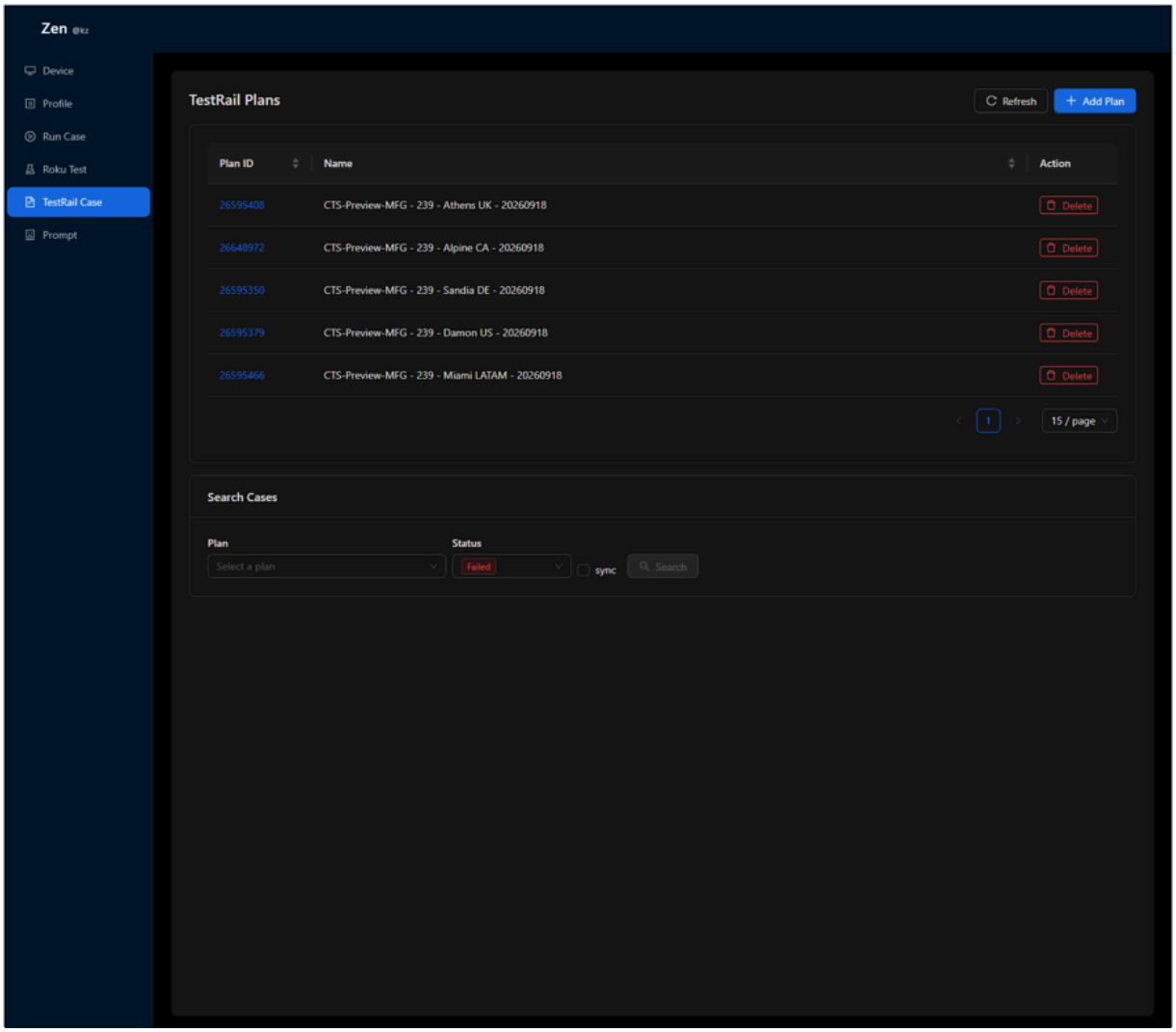
[Screenshot: Live log panel — dark console with red error lines and amber warnings.]

5.3 Test Logs

Past logs are listed with filters for **device**, **case id**, **result**, and **date**. Columns: **Log File**, **State** (running / done), **Result** (Pass / Fail / Skipped / Blocked, color-coded), and **Action** (**View** opens the log; **Delete** removes it).

6. TestRail Case Tab

Manage the TestRail plans you care about, and search their test cases by status. Results are cached so they can feed the "Add Profile with RunPlan" generator.



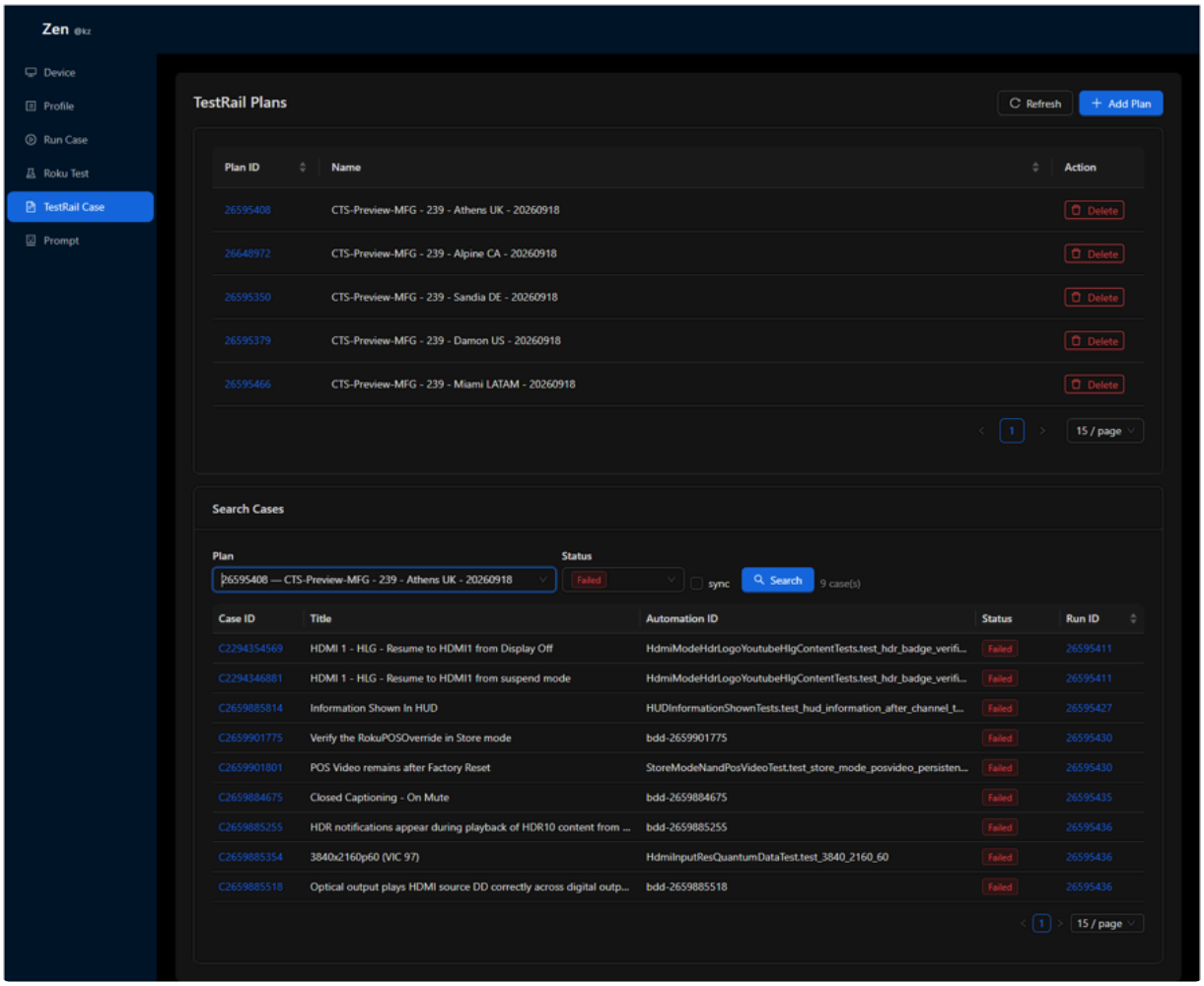
6.1 Plans

- The **TestRail Plans** table lists saved plans: **Plan ID** (links to TestRail), **Name**, and a **Delete** action.
- Click **Add Plan**, enter the **Plan ID** (e.g. `26595408`) and a **Name**, and save.

6.2 Searching cases

1. In the **Search Cases** card, pick a **Plan** (shown as *id — name*).
2. Pick a **Status** — each status is shown as a colored tag (e.g. Failed = red, Passed = green, Retest = gold, For Review = blue).
3. Optionally tick **sync** to force a fresh fetch from TestRail (bypasses the cache).
4. Click **Search**.

Results table columns: **Case ID** (links to TestRail, shown as `C<id>`), **Title**, **Automation ID**, **Status** (colored tag), and **Run ID** (sortable, links to the TestRail run).

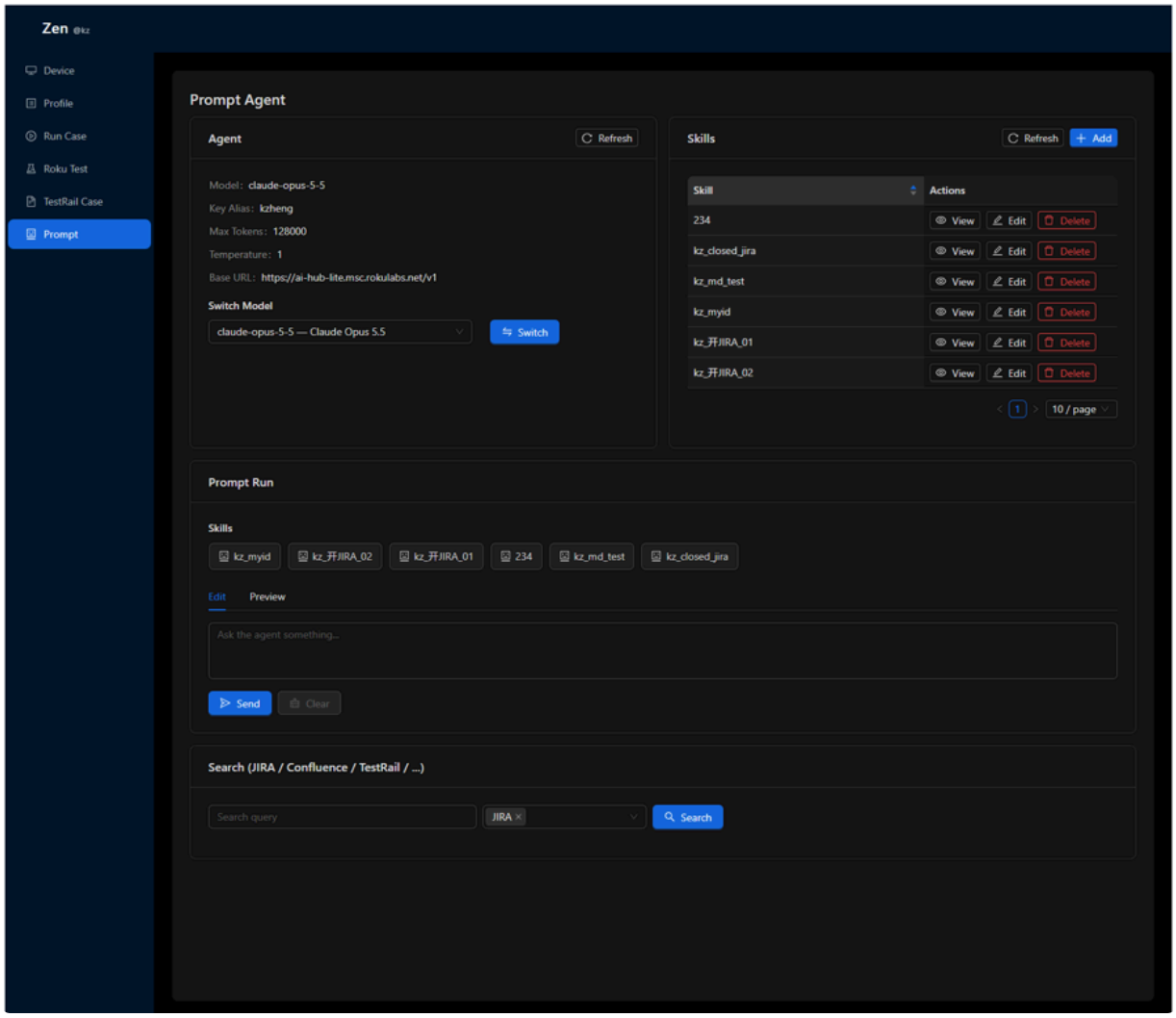


6.3 Feeding the profile generator

Running a search caches the results as `<plan>_<status>.json`. Once cached, that plan+status becomes selectable in **Profile → Add Profile with RunPlan** (§3.2).

7. Prompt Tab (AI Assistant)

The Prompt tab is an AI assistant that can answer questions using **Roku internal data** (JIRA, Confluence, TestRail, GitLab) via retrieval-augmented generation (RAG). It also lets you save reusable prompt **Skills**.



7.1 Agent card — model info & switching

Shows the active **Model**, **Key Alias**, **Max Tokens**, **Temperature**, and **Base URL**. To change models, pick one from the **Switch Model** dropdown and click **Switch**. Available models include Claude (Opus/Sonnet 4.x–5.x) and GPT (5.x/6) families.

7.2 Skills card

A **Skill** is a saved block of Markdown (a reusable prompt fragment). The card lists all skills with **View**, **Edit**, and **Delete** actions, plus **Add**.

- **View** renders the skill as formatted Markdown.
- **Add / Edit** opens a dialog with a **Name** and a Markdown editor that has **Edit** and **Preview** tabs.

[Screenshot: Skill Add/Edit dialog — Edit / Preview tabs over a Markdown body.]

7.3 Prompt Run

1. **Skills** — each saved skill appears as a clickable chip. Click a chip to **append its content** into the prompt box (the chip highlights). Click several to combine them; click again to append

again.

2. **Prompt** — type your question. The input has **Edit** and **Preview** (Markdown) tabs and grows with your content.
3. Click **Send**. Zen runs with internal-data access on: it first tries to turn your question into a JIRA query (JQL) and, if that does not apply, falls back to semantic search across the selected sources — then answers using what it found.
4. Use **Clear** to reset the prompt, active skills, and the result.

The answer is rendered as Markdown. Two buttons sit below it: **Save** (downloads the answer as a `.md` file — in Chrome/Edge a native "Save As" dialog lets you choose the location) and **Copy** (copies the answer to the clipboard).

[Screenshot: Prompt Run — a question, skill chips highlighted, and a rendered Markdown answer with Save / Copy buttons.]

> Questions like "list the JIRA tickets I closed last week" work well — Zen recognizes the structured intent and runs a real JQL query. For topic lookups, include English keywords or a ticket prefix (e.g. `AMLOGIC`) for better recall.

7.4 Search card

A direct semantic search over indexed internal data. Enter a **query**, pick one or more **Sources** (JIRA, Confluence, TestRail, GitLab, File, S3, Web — default JIRA), and click **Search**. Results list the **Document** (linked) and a **Content** preview.

[Screenshot: Search card — query box, Sources multi-select, and a results table of documents.]

8. Tips & Troubleshooting

Symptom	What to do
A tab is missing from the menu	It was disabled in `frontend/conf/config.yml` (`tabs:`). Re-enable it and restart the frontend.
Copy button does nothing	The clipboard API needs a secure context. Over plain `http://<ip>` Zen falls back to a legacy copy; if it still fails, use Ctrl+C on the selected text.

Save doesn't show a folder picker	The native "Save As" picker is Chromium-only and needs HTTPS/localhost. Otherwise the file just downloads to your default folder.
A plan/status is not selectable in "Add Profile with RunPlan"	Search that plan+status in the TestRail Case tab first to cache its data.
The assistant says it can't reach internal data	The internal search gateway had a transient error. Retry the prompt.

Zen User Guide — generated from source at `QATools` *and the live instance http://10.64.224.182:9898/.*